



UE23AM342BA3
Explainable AI

INTERPRETABILITY FOR UNSUPERVISED LEARNING & NLP

Unit: 4

Prof. Ayisha Noori VK

Dept. of CSE(AI&ML), PES University, Bengaluru



Pre-model Explainability - Understanding the "ingredients" before you start cooking the "meal"

Explainability applied before training analyzing raw data structure, distributions, and feature relationships to guide model selection and preprocessing.

What is Pre-model XAI?

- Applies explainability BEFORE any model is trained
- Goal: understand raw data structure, distributions, and relationships
- Helps select the right model family and preprocessing steps
- Catches hidden biases, outliers and data-quality issues early
- Reduces post-hoc debugging by surfacing problems at source
- Encompasses Exploratory Data Analysis EDA, visualization, and statistical summary methods

Why It Matters

- Detects class imbalance before training skews model learning
- Ensures transparency in the data pipeline itself
- Allows domain experts to review data before modelling begins
- Reduces costly retraining when biases are caught early
- Directly shapes downstream model interpretability and fairness
- Satisfies regulatory requirements for data documentation (General Data Protection Regulation (GDPR) - GDPR, AI Act)

Key Techniques

- Exploratory Data Analysis EDA: histograms, box plots, pair plots for feature distributions ✓
- PCA / t-SNE / UMAP: dimensionality reduction for 2D/3D visualization ✓
- Correlation matrices and mutual information for feature relationships ✓
- Missing-value heatmaps: detect data sparsity patterns ✓
- Feature importance proxies via simple logistic/linear regression ✓
- Data provenance tracking: record transformations for audit trails ✓

Unsupervised Learning for Clustering

Clustering algorithms group unlabeled data by similarity. Explainability means making cluster structure, boundaries, and assignments human-interpretable.

Overview of Clustering

- Groups unlabeled data by intrinsic similarity — no ground-truth labels
- ✓ • K-Means: minimizes within-cluster sum of squares (WCSS)
- ✓ • DBSCAN: density-based special clustering of application with noise ; discovers arbitrary-shaped clusters
- ✓ • Agglomerative and divisive - Dendrogram: hierarchical merging by linkage criterion
- GMM – Gaussian mixture model / expectation maximization model: probabilistic; assigns soft cluster memberships
- Evaluation: Silhouette score, Davies-Bouldin, Calinski-Harabasz

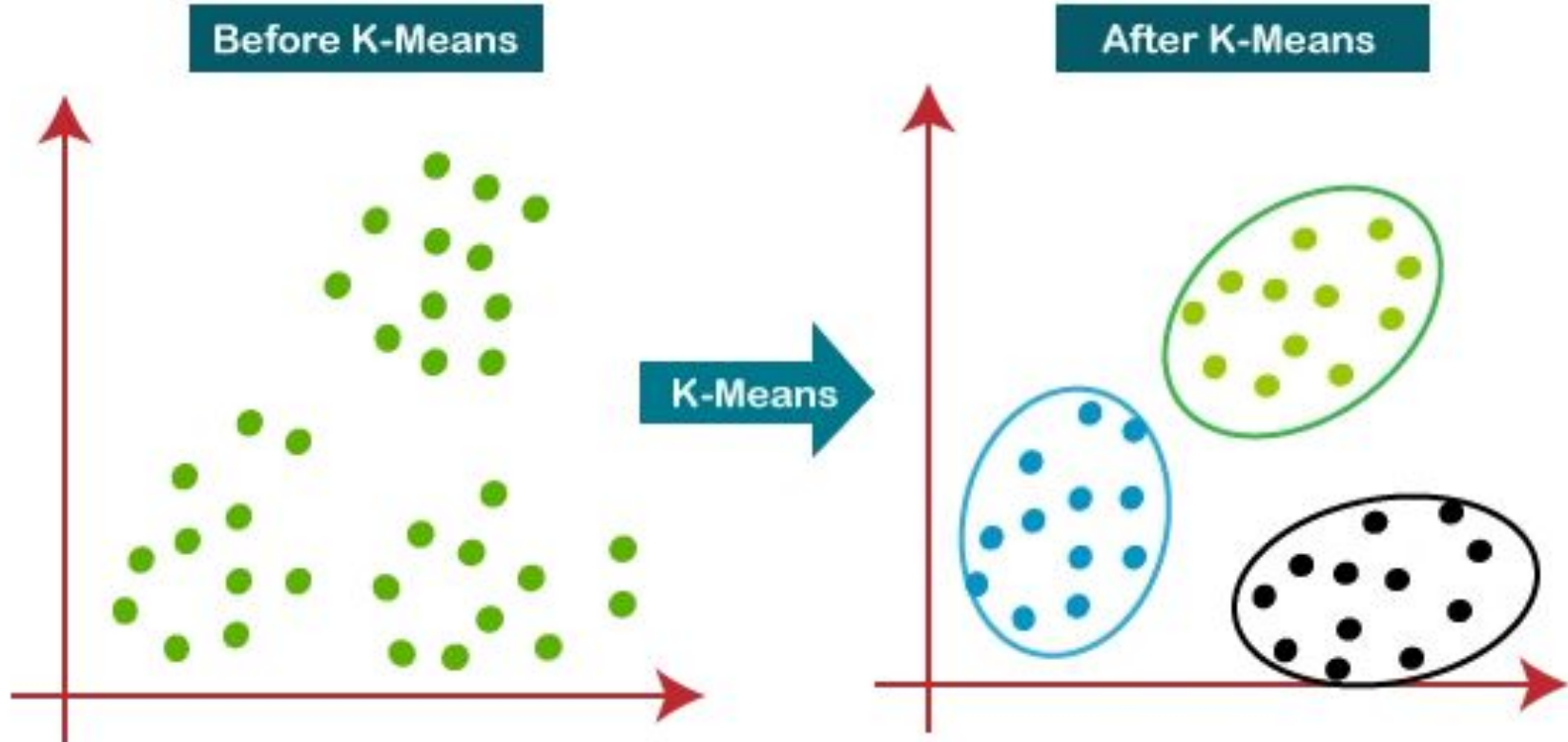
Explainability Challenges

- Clusters are unlabeled — must assign semantic meaning post-hoc
- High-dimensional data makes decision boundaries opaque
- K-Means is non-deterministic; different runs may produce different clusters
- No single 'correct' clustering — interpretation is subjective
- Evaluation metrics don't capture human-meaningful cluster structure
- Noise points (DBSCAN) complicate holistic interpretation

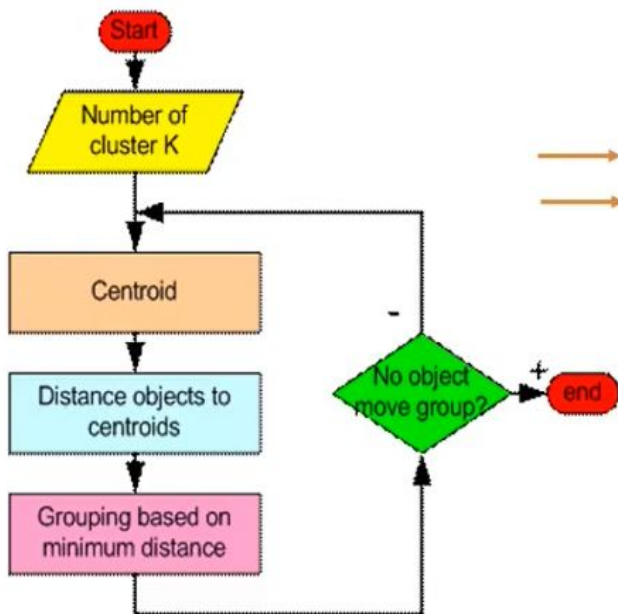
Explainability Approaches

- t-SNE / UMAP: project high-dimensional clusters into 2D scatter plots
- Centroid profiling: summarize each cluster by its average feature values
- Surrogate decision tree: train tree on (data, cluster label) pairs
- SHAP values: attribute cluster membership to individual features
- Prototype & criticism: find representative and atypical samples per cluster
- Human-in-the-loop: domain expert labels discovered clusters

K Mean clustering



K Mean clustering



Height	Weight
185	72
170	56
168	60
179	68
182	72
188	77
180	71
180	70
183	84
180	88
180	67
177	76

Cluster 1 (185, 72)

Cluster 2 (170, 56)

Euclidean Distance

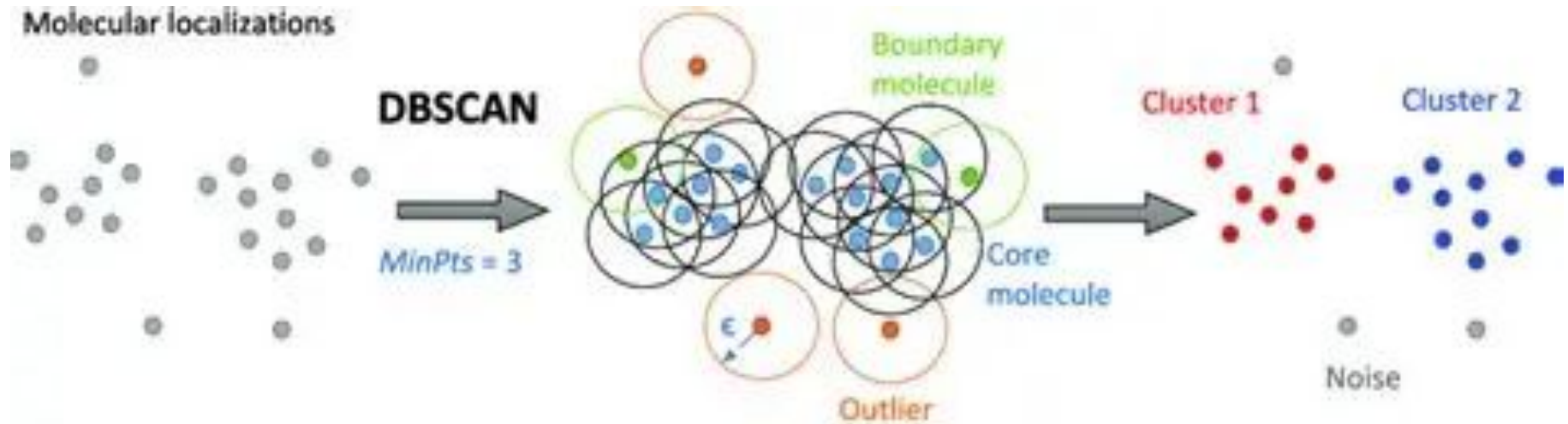
$$\sqrt{(x_H - C_1)^2 + (x_W - C_2)^2}$$

$$\sqrt{(168 - 185)^2 + (60 - 72)^2} = 20.80$$

$$\sqrt{(168 - 170)^2 + (60 - 56)^2}$$

Please open your IDEs and let's design technology demonstrator for this concept

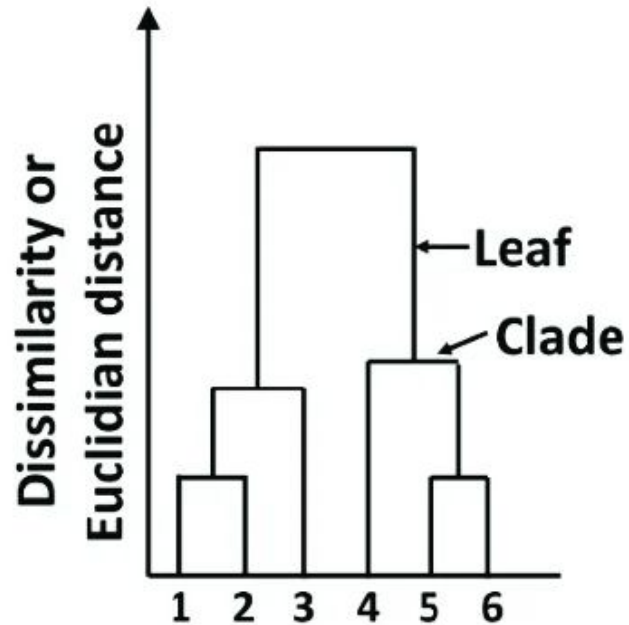
DBSCAN: density-based special clustering of application with noise clustering



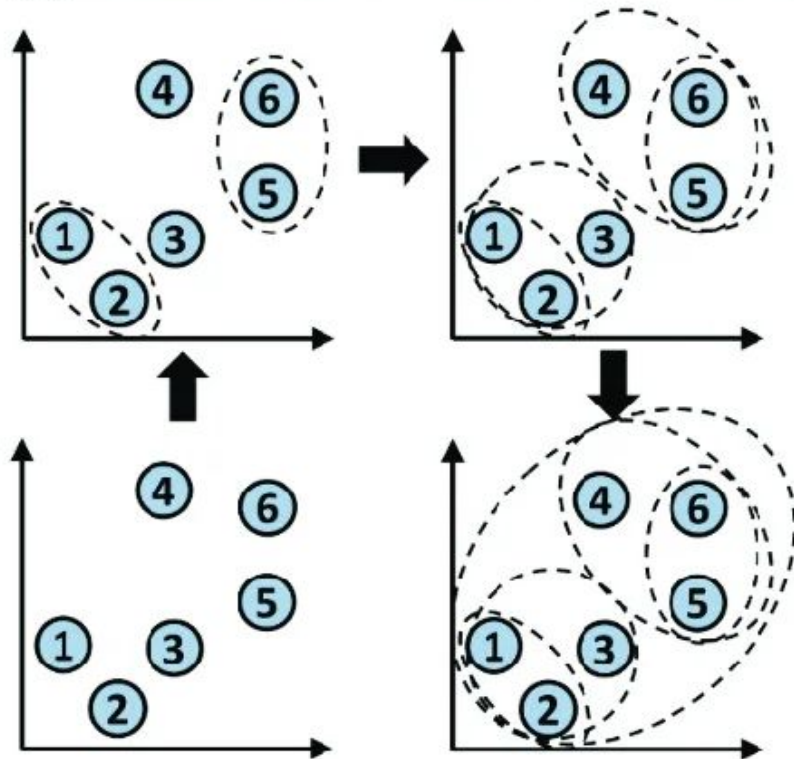
Core point
Boundary point
Epsilon

Please open your IDEs and let's design technology demonstrator for this concept

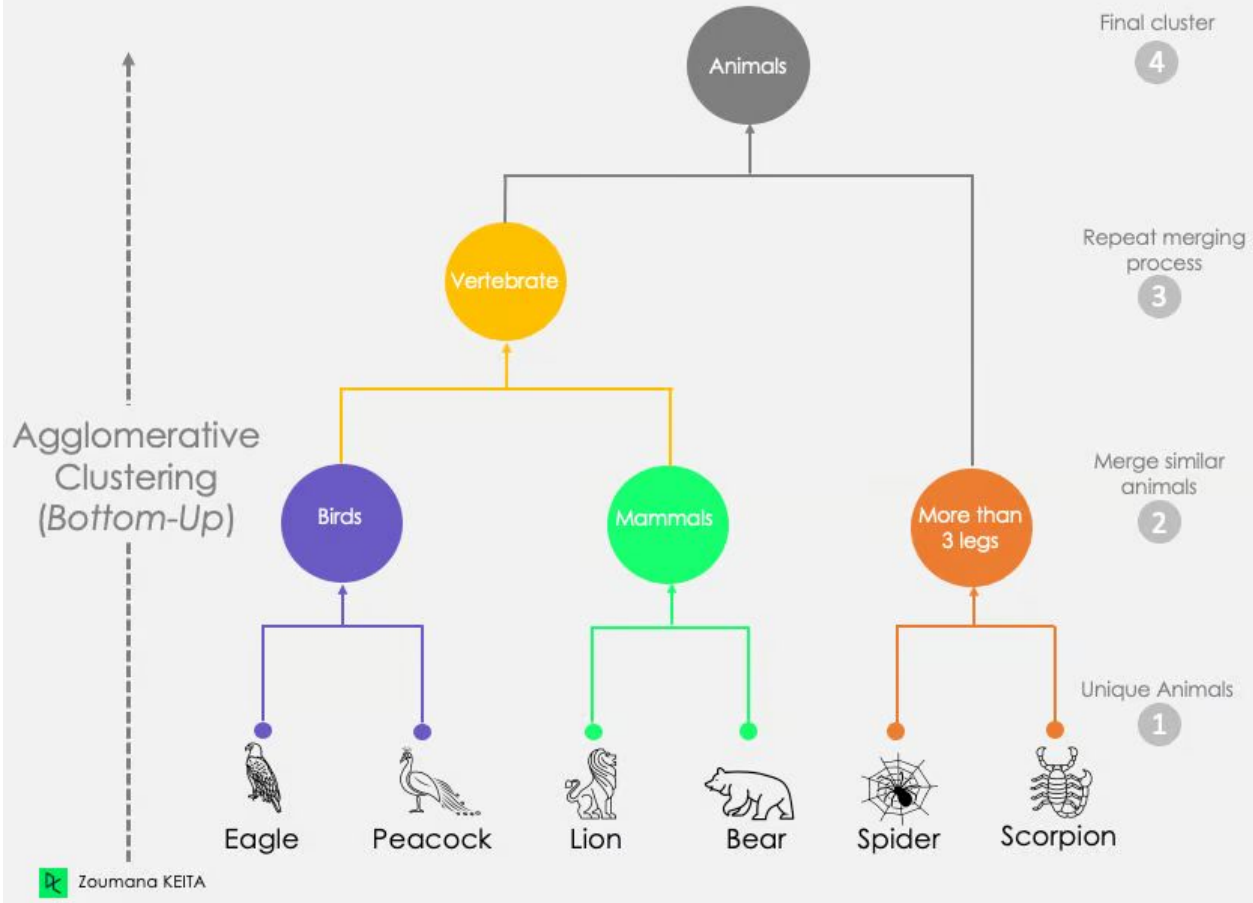
Dendrogram



Agglomerative hierarchical clustering

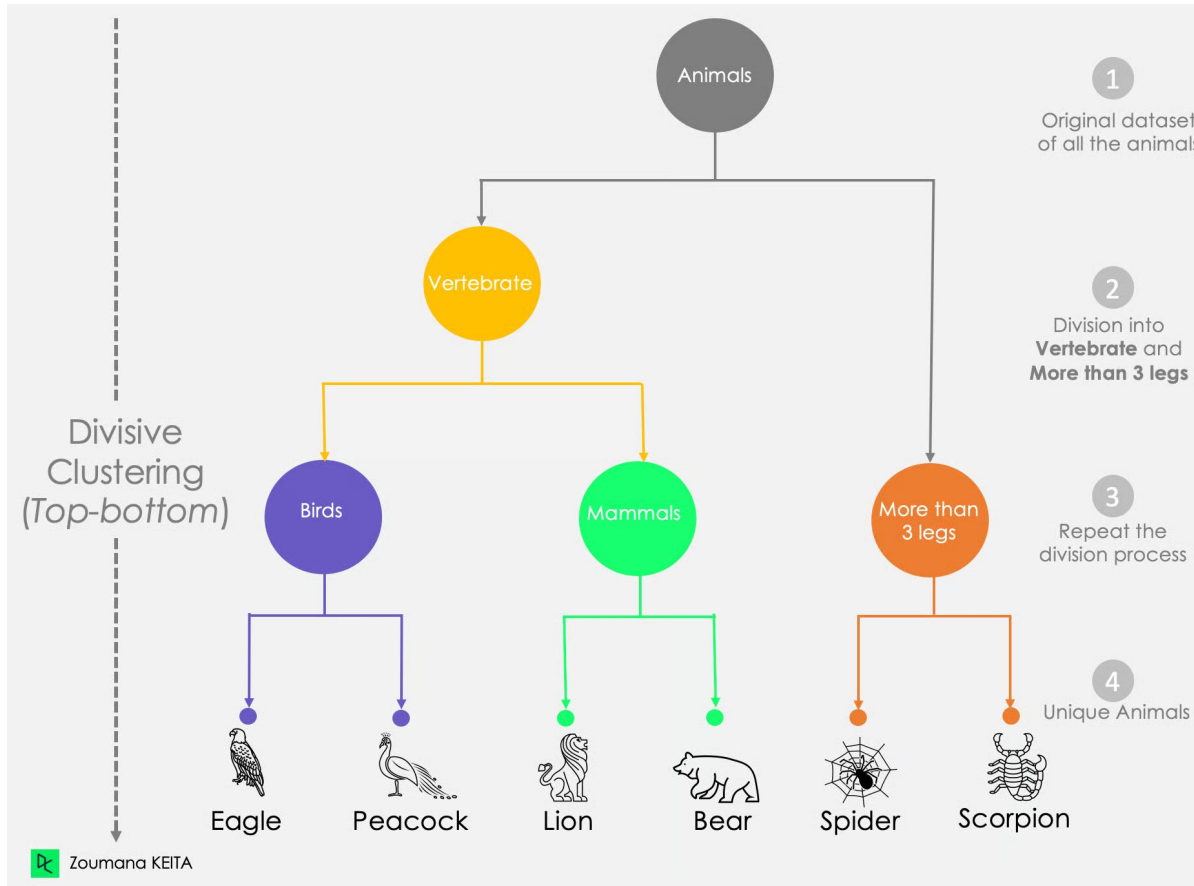


Agglomerative Clustering

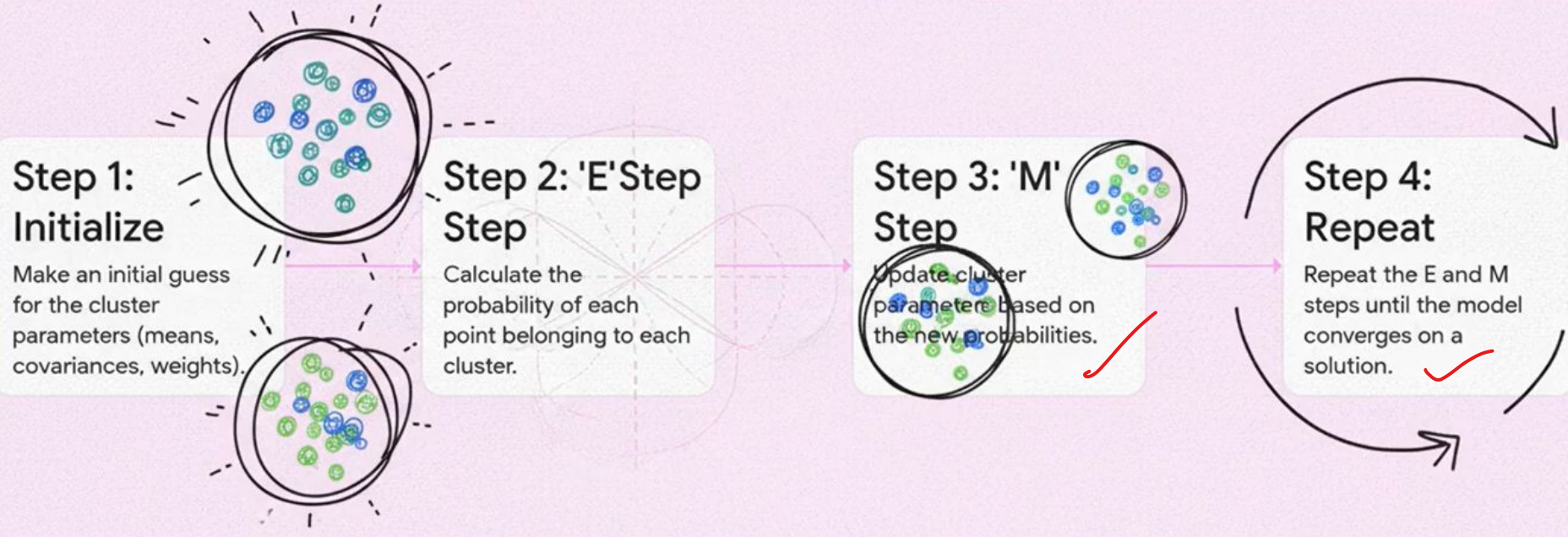


Please open your IDEs and let's design technology demonstrator for this concept

Divisive Clustering



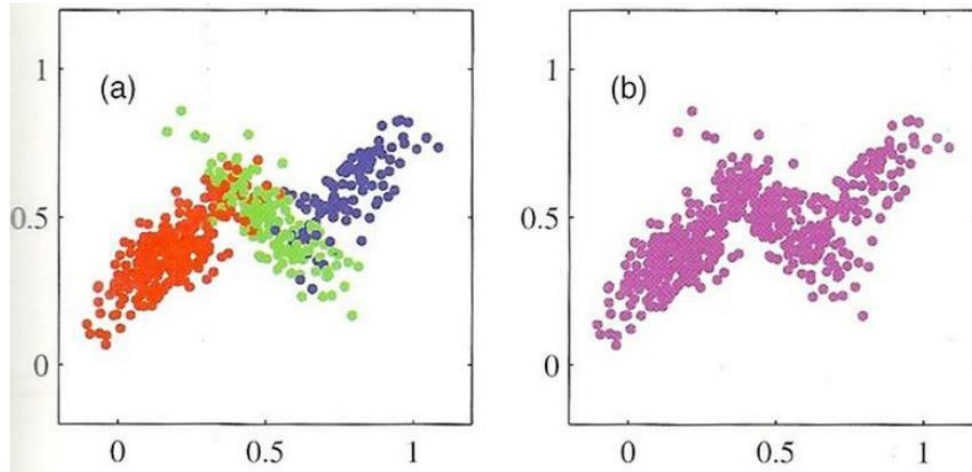
GMM – Gaussian mixture model / expectation maximization model clustering



GMM – Gaussian mixture model / expectation maximization model clustering



Gaussian Mixture Models



Please open your IDEs and let's design technology demonstrator for this concept

Summary of Unstructured Data

Unstructured data (text, images, audio) lacks predefined schema. Interpretability requires specialized tools bridging raw inputs and model decisions.

Types of Unstructured Data

- Text: reviews, news articles, clinical notes, legal documents
- Images: photographs, X-rays, satellite imagery, pathology slides
- Audio: speech, music, environmental sensor signals
- Video: sequences of frames + temporal dependencies
- Mixed modality: image-caption pairs, video + transcripts
- Graph data: social networks, molecular structures, knowledge graphs

Challenges for XAI

- No predefined feature schema — must extract features first
- Extremely high dimensionality: millions of pixels, thousands of tokens
- Semantic gap: raw pixels/tokens are far from human concepts
- Distributed representations encode meaning across many neurons ✓
- Domain knowledge required to validate whether explanations make sense
- Explanations must be modality-appropriate (visual vs. textual)

Interpretability Methods

- **Saliency maps / gradient-based**: pixel-level attribution for images
- **Attention visualization**: attended tokens in transformer NLP models
- **LIME superpixel masks**: highlight image regions driving predictions
- **SHAP**: exact Shapley values for NLP and vision pipelines
- **TCAV / ACE**: TCAV (Testing with Concept Activation Vectors) and ACE (Automated Concept-based Explanation) are concept-based explanations using hidden layer probing
- **GradCAM**: class-activation maps localized to convolutional layers

K-Means Clustering Explainer

K-Means partitions data into K clusters by minimizing within-cluster variance. Explaining K-Means reveals cluster composition, boundaries, and feature contributions.

How K-Means Works

- Step 1: Choose K — the number of clusters (use elbow plot or silhouette)
- Step 2: Initialize centroids randomly or via K-Means++ (better coverage)
- Step 3: Assign each point to nearest centroid using Euclidean distance
- Step 4: Recompute centroids as the mean of all assigned points
- Step 5: Repeat steps 3-4 until centroids no longer move (convergence)
- Objective: minimize WCSS (Within-Cluster Sum of Squares) = $\sum \sum ||x_i - \mu_k||^2$ over all clusters

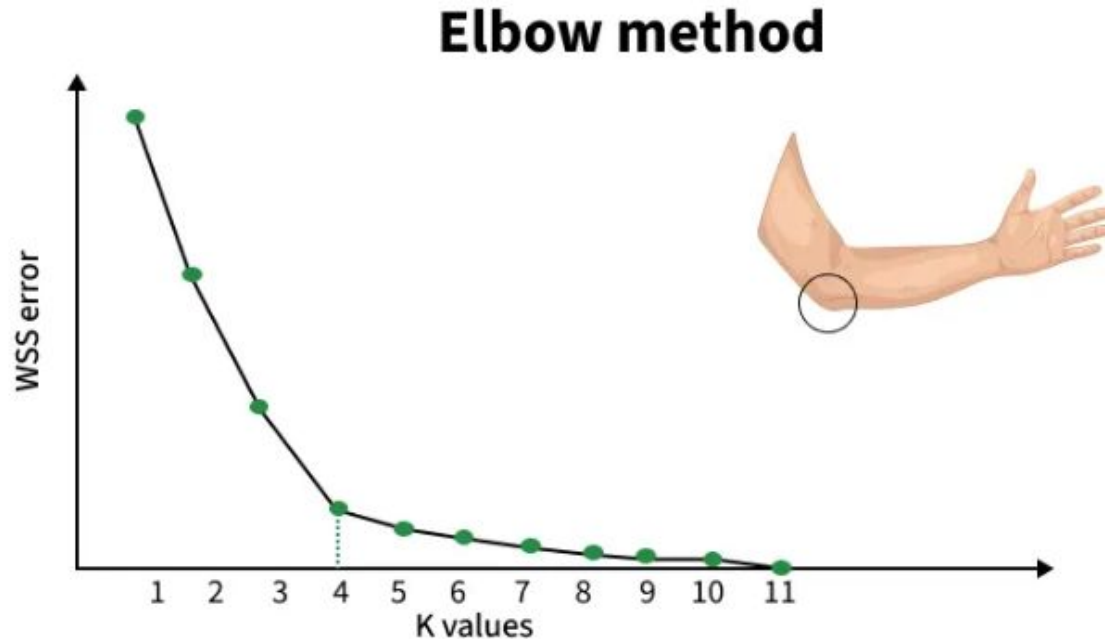
Explaining Cluster Assignments

- **Centroid profiles:** tabulate mean feature values per cluster for comparison
- Surrogate decision tree: fit tree on (X, cluster_labels) to get if-then rules
- Prototype selection: find real samples closest to each centroid
- Criticisms (MMD-critic): find samples poorly represented by prototypes
- SHAP cluster explainer: assign per-feature credit for cluster membership
- Parallel coordinate plots: visualize how each feature separates clusters

Evaluation & Visualization

- **Elbow plot:** plot WCSS (Within-Cluster Sum of Squares) vs K; look for the 'elbow' (diminishing returns)
- **Silhouette score:** ranges from -1 to 1; higher = better-defined clusters
- **Davies-Bouldin index:** lower = more compact and well-separated clusters
- **t-SNE / PCA scatter plot:** 2D view with color-coded cluster labels
- **Heatmap:** feature averages per cluster — reveals cluster profiles at a glance
- **Radar (spider) chart:** multi-axis comparison of cluster centroid features

The Elbow Method helps by plotting the Within-Cluster Sum of Squares (WCSS) against increasing k values and looking for a point where the improvement slows down, this point is called the "elbow."



The Silhouette Score is a metric used to evaluate the quality of clustering results. It measures how similar each data point is to its own cluster compared to other clusters, helping assess how well the data has been grouped.

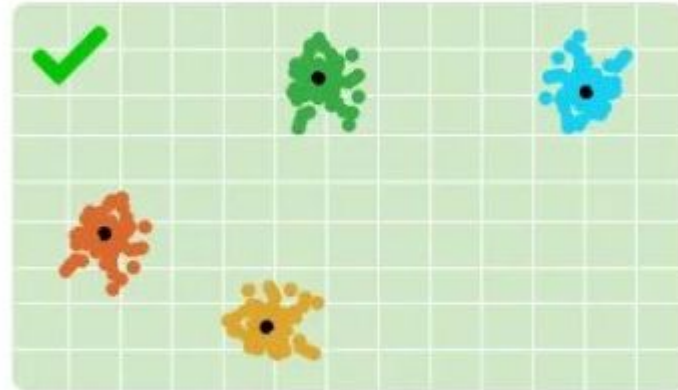
Visual Comparison of Clustering with Different Centroids and Their Silhouette Scores

K-Means (Six Centroids)



Silhouette Score: 0.58

K-Means (Four Centroids)

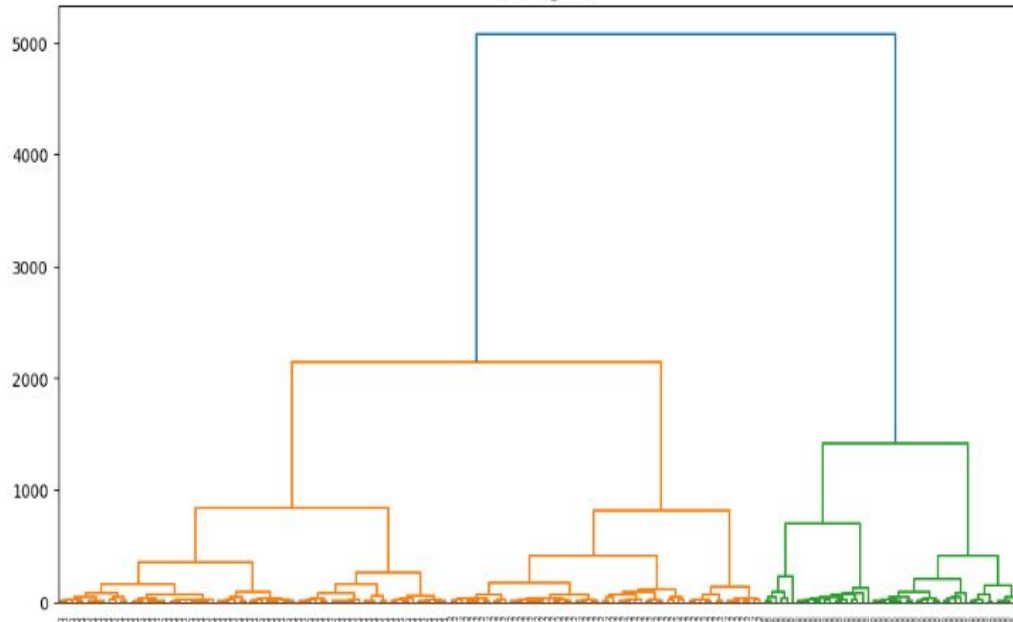


Silhouette Score: 0.84
Higher score, better clustering

The Davies-Bouldin Index enables to assess the clustering quality by comparing the average similarity between the pairwise most similar clusters

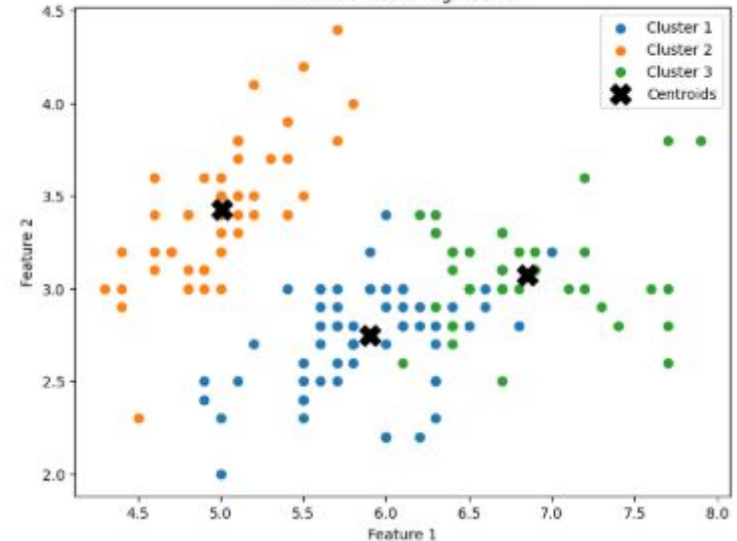
Davies-Bouldin Index: 0.5357343073560251

Dendrogram



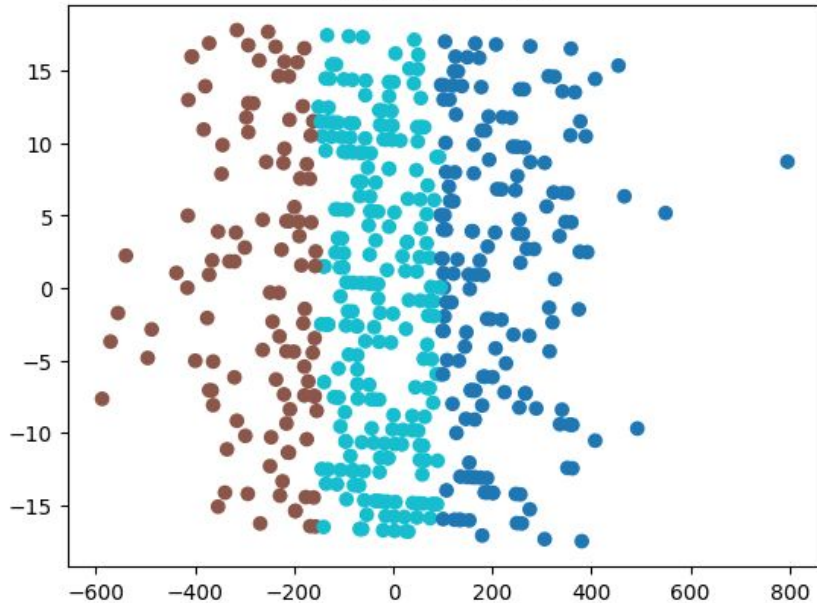
Davies-Bouldin Index: 0.6619715465007465

K-Means Clustering Results

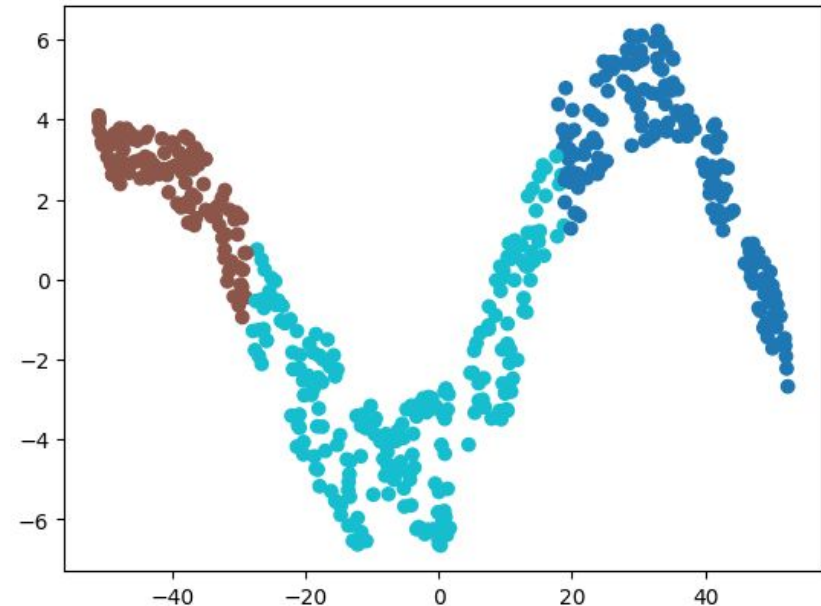


t-Distributed Stochastic Neighbor Embedding (t-SNE) is a technique for dimensionality reduction that is particularly well suited for the visualization of high-dimensional datasets

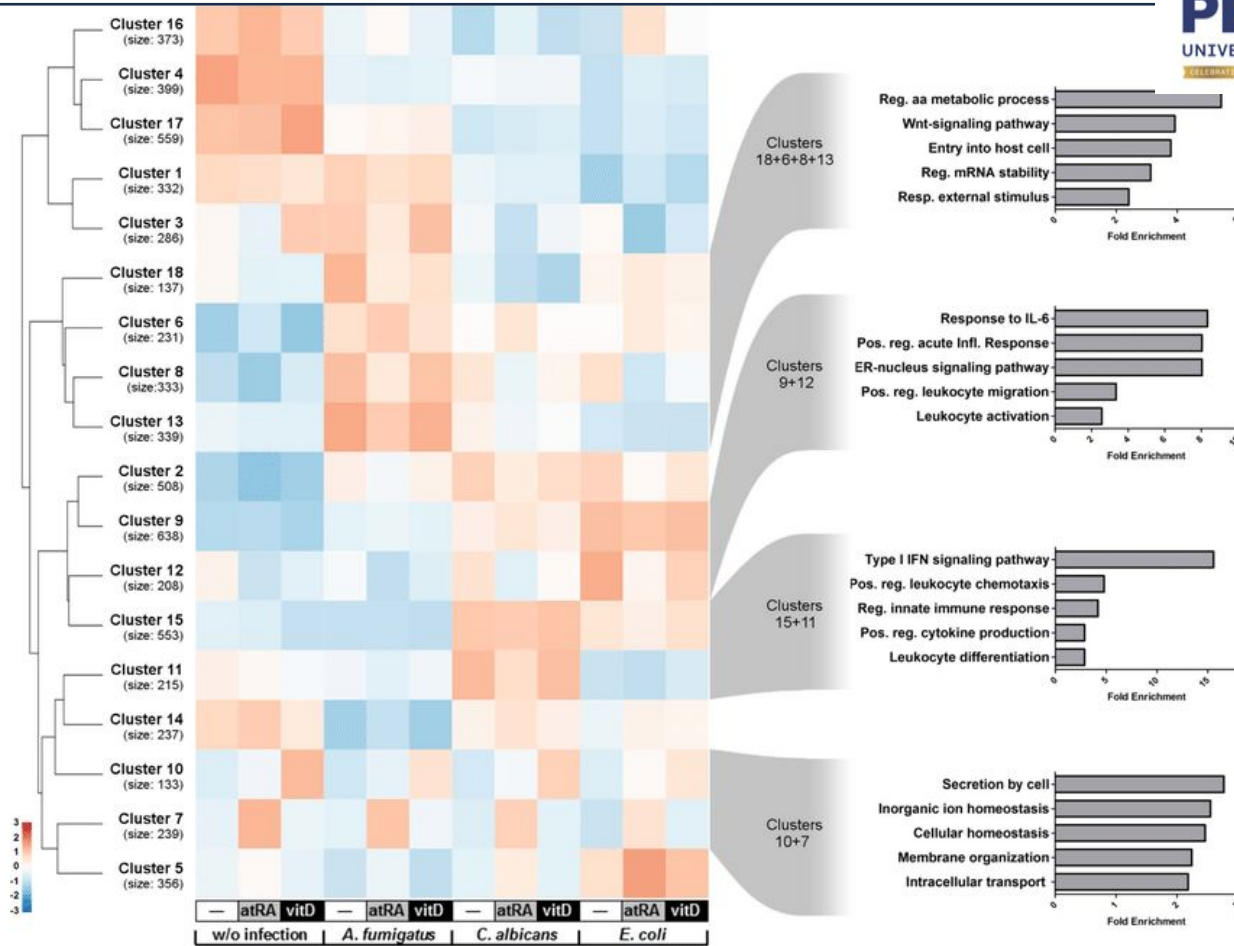
PCA Scatter Plot



t-SNE Scatter Plot

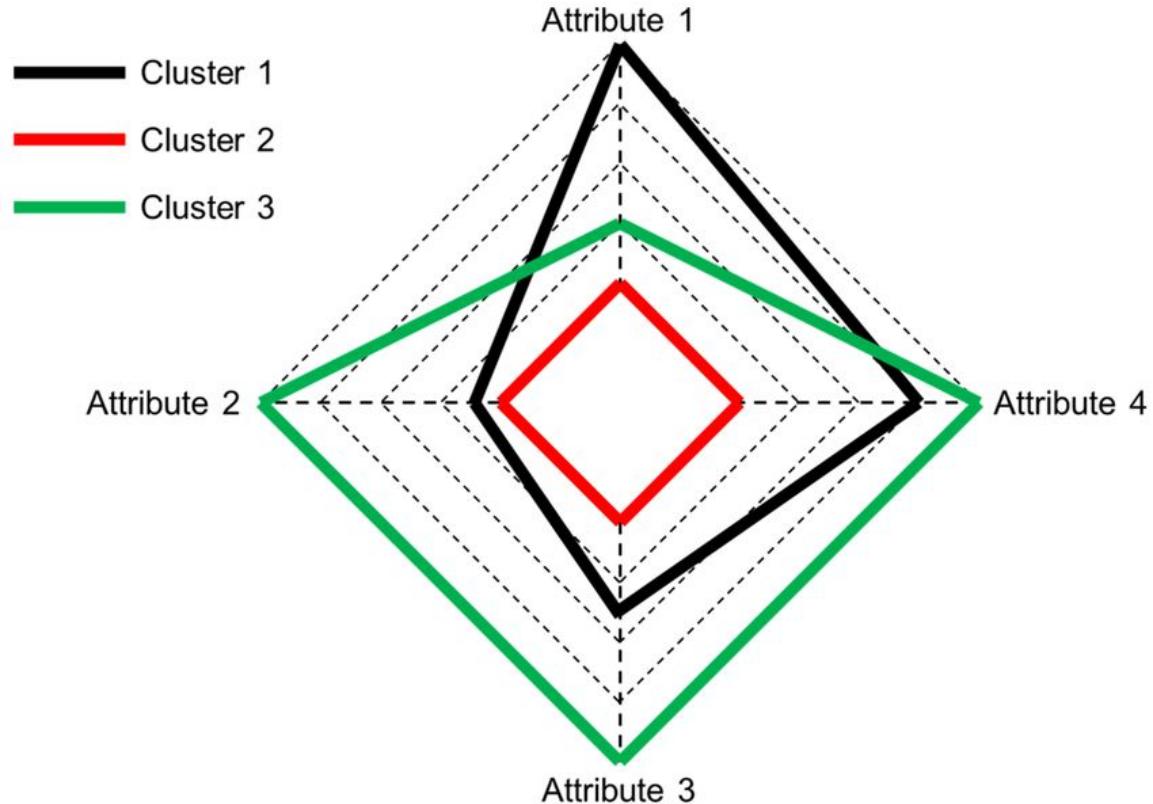


Heatmap of K-means clustering analysis can be performed on variance-stabilized read counts to build a heatmap for the differentially expressed parameters. The clustering of the rows is based on euclidean distance. The colors in the map represent rowscaled expression levels: blue indicates the lowest expression, white indicates intermediate expression, and red indicates the highest expression.



Radar chart by gamified class data

Radar chart based on K means analysis is aimed to investigate the efficacy of learning gamification in developing sustainable educational environments to parameter patterns.



NLP: Attention and Concept-Based Explanations

Transformer NLP models use attention mechanisms that can be visualized. Concept-based methods (TCAV, ACE) offer higher-level, human-understandable explanations.

Attention as Explanation

- Self-attention scores quantify which tokens influence each other
- Multi-head attention: each head captures different linguistic patterns
- Attention rollout: propagate attention weights across all layers
- BertViz: interactive visualization of all attention heads in BERT/GPT
- **Ecco: token attribution via input reduction in generative models**
- Key limitation: high attention $\neq$ high gradient importance; not always faithful

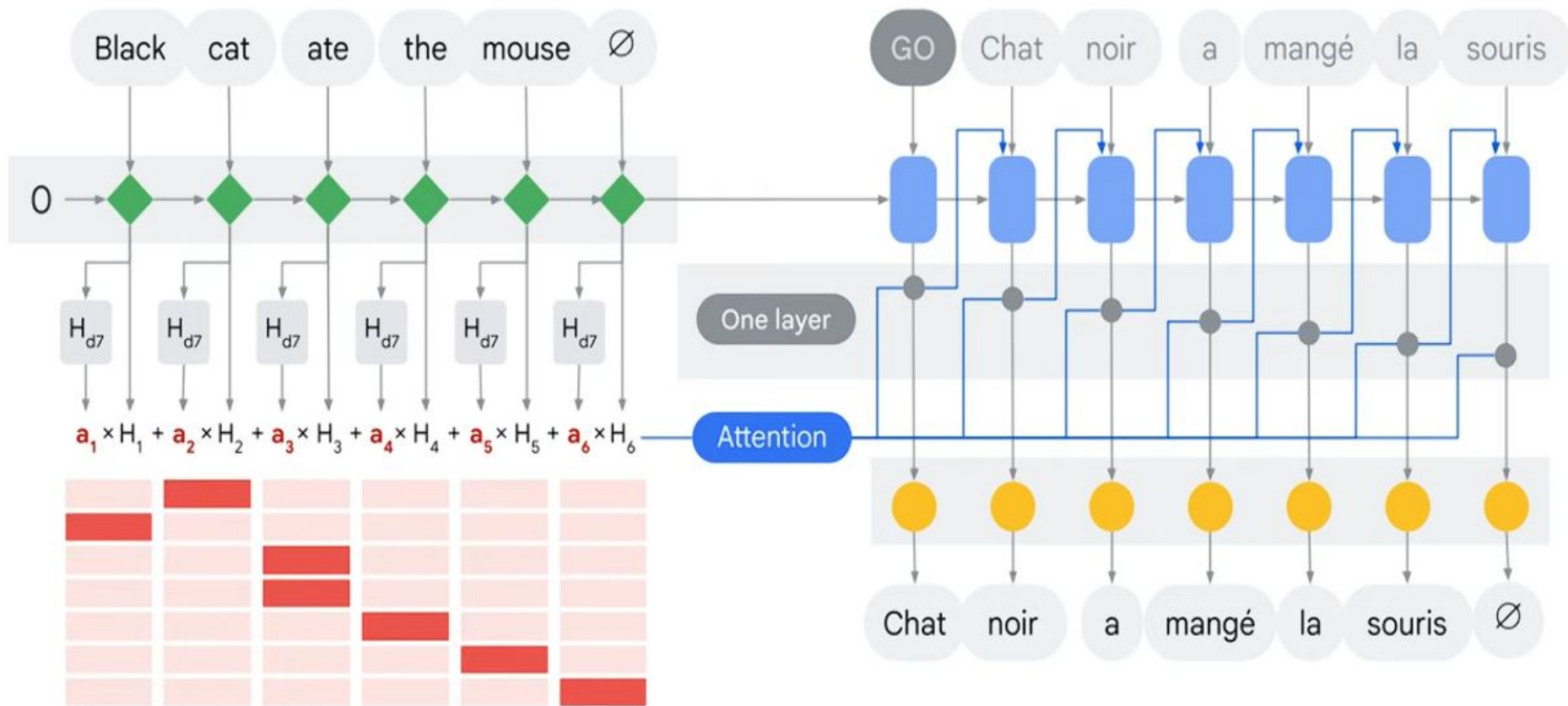
Concept-Based Explanations

- TCAV: trains linear probe on concept images; computes sensitivity score
- ACE: clusters activation patches unsupervised to discover concepts
- Concept probing: test if hidden states encode properties (POS, NER, sentiment)
- ConceptSHAP: SHAP-based concept attribution for structured explanations
- Polarity / negation / entity type emerge as linear-separable directions
- Domain experts validate model behavior using familiar vocabulary

Applications & Insights

- **Sentiment:** identify which words flip polarity in BERT predictions
- **Bias detection:** reveal gender/racial bias in coreference resolution
- **QA models:** trace which passage spans evidence the generated answer
- **Hallucination debugging:** attention divergence signals unreliable generation
- **Regulatory compliance:** provide human-readable audit trails for NLP decisions
- **Active learning:** focus annotation on high-uncertainty, low-attention samples

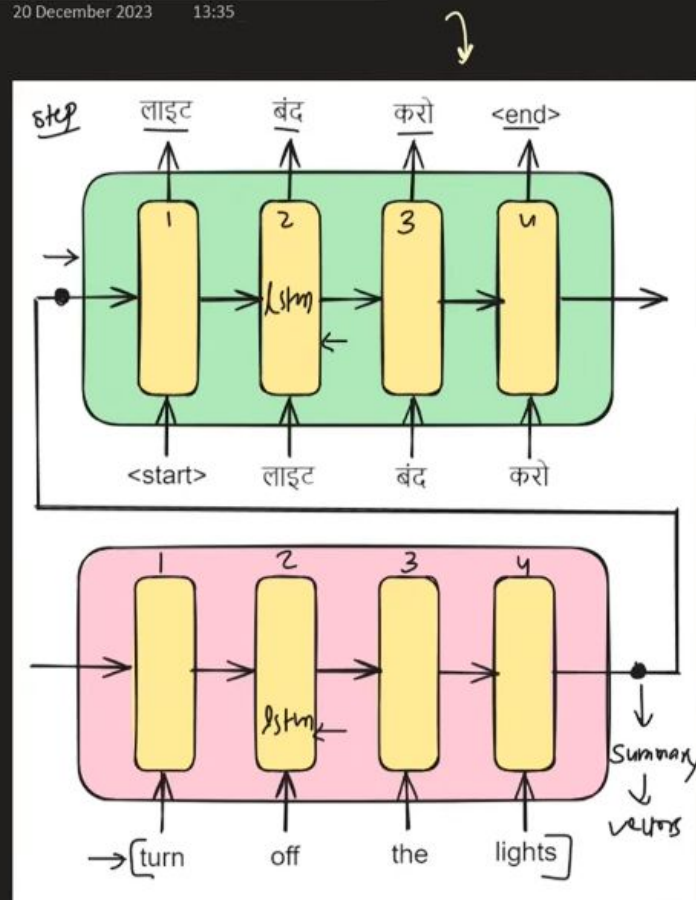
Improve translation with attention network



The Why

20 December 2023

13:35



50 words

Once upon a time in a small Indian village, a mischievous monkey stole a turban from a sleeping barber, wore it to a wedding, danced with the bewildered guests, accidentally got crowned the 'Banana King' by the local kids, and ended up leading a vibrant, impromptu parade of laughing villagers, cows, and street dogs, all while balancing a stack of mangoes on its head, creating a hilariously unforgettable spectacle and an amusing legend that the village still chuckles about every monsoon season.

Explaining Sentiment Analysis

Sentiment analysis classifies text as positive, negative, or neutral. Explanations reveal which words and constructs drive the model's output.

Sentiment Model Overview

- **Lexicon-based:** VADER, SentiWordNet — polarity lookup dictionaries
- **ML-based:** Logistic Regression / SVM on TF-IDF feature vectors
- **Deep learning:** fine-tuned BERT, RoBERTa, DistilBERT on SST-2, IMDb
- **Aspect-level:** targets sentiment toward specific entities in text
- Output: polarity label (Positive / Negative / Neutral) + confidence score
- Ensemble: combine lexicon + neural for improved robustness

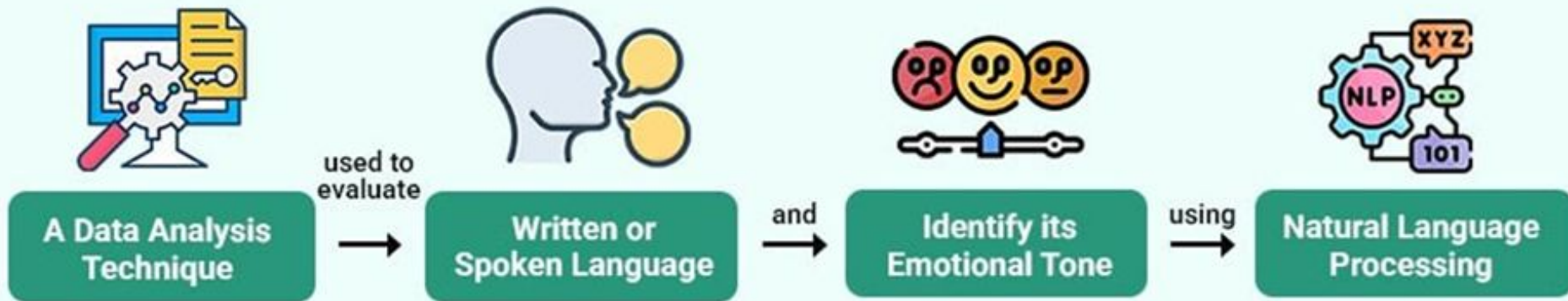
Explanation Methods

- LIME: perturb input tokens; fit local linear model around prediction
- SHAP (KernelSHAP): model-agnostic Shapley values per token
- Integrated Gradients: path-integral attribution from blank baseline to input
- Attention visualization: highlight attended tokens in transformer layers
- Anchors: find minimal word sets that 'lock in' the predicted class
- Contrastive explanation: show minimal edit that flips prediction

Challenges & Best Practices

- **Negation:** 'not good' fails naive attribution — needs phrase-level analysis
- **Sarcasm and irony** defeat surface-level word polarity methods
- **Domain shift:** 'sick' = bad in general, = very good in slang
- **Explanation faithfulness:** compare LIME / SHAP against gradient attribution
- **Human evaluation:** check if explanations align with linguistic intuition
- **Stability testing:** run explanation method 5+ times; flag high variance

What is Sentiment Analysis?



Types:



Layer Integrated Gradients

Integrated Gradients (IG) attributes predictions to input features by integrating gradients along a path from a baseline to the actual input.

Core Concept

- Axiom-based method satisfying Completeness and Sensitivity properties
- Completeness: attributions sum to $F(x) - F(\text{baseline})$ — no missing credit
- Sensitivity: if baseline and input differ only in feature i , $\text{attr}_i \neq 0$
- Baseline choice matters: zero vector, mask token, or average input
- Layer IG: applies IG to internal layer activations, not raw inputs
- Captures how intermediate representations contribute to the prediction

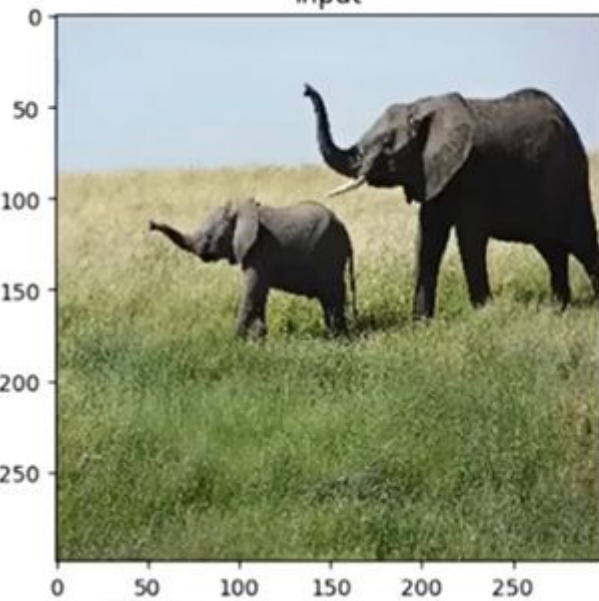
Mathematical Formulation

- $\text{IG}_i(x) = (x_i - x'_i) \times \int_{[0,1]} (\partial F(x' + \alpha(x - x')) / \partial x_i) d\alpha$
- Straight-line path from baseline x' to input x in feature space
- Approximated via Riemann sum: average gradient over ~ 50 steps
- Layer IG: replace x with layer activation $h_L(x)$ in the same formula
- Available in Captum (PyTorch): `LayerIntegratedGradients` class
- Also supported in TensorFlow via `tf-explain` and `Alibi` library

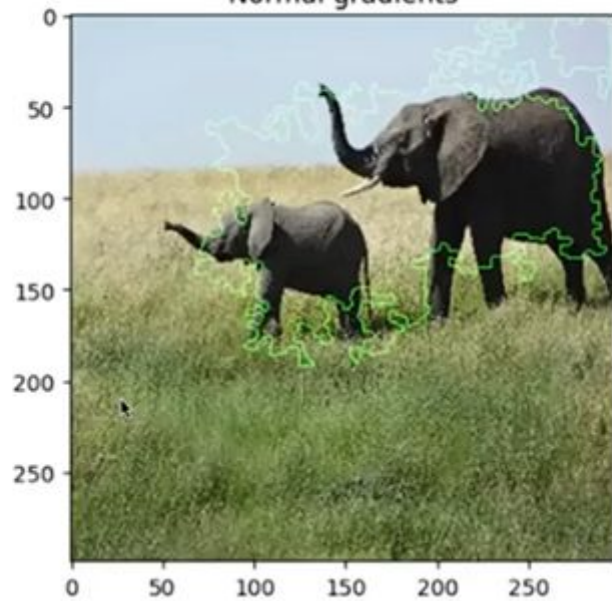
Applications in NLP

- Token-level attribution: which input words drive BERT/GPT classification
- Embedding layer IG: reveals which token embeddings carry most signal
- Intermediate layer analysis: traces concept formation through layers
- Debugging: find why model misclassifies adversarial or ambiguous text
- Compare IG vs attention: IG is gradient-based and more mathematically faithful
- Visualize as colored token highlight bar (green = positive, red = negative)

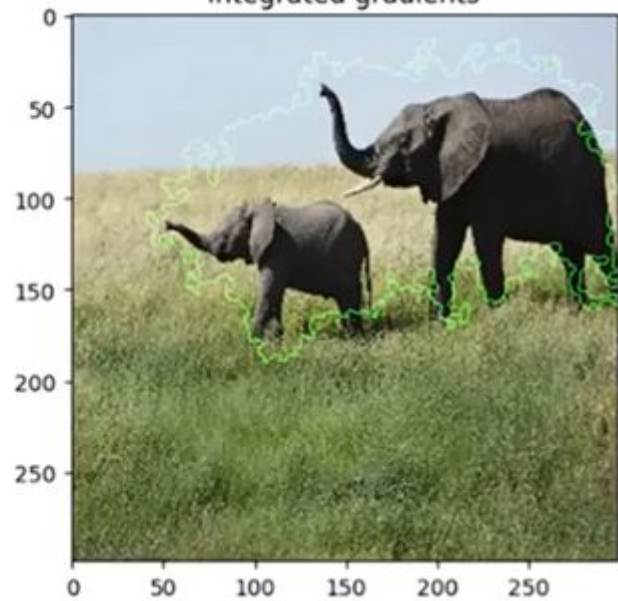
Input



Normal gradients



Integrated gradients



Using LIME for Sentiment Analysis

LIME (Local Interpretable Model-agnostic Explanations) explains any classifier locally by fitting a simple interpretable model around a specific prediction.

LIME Algorithm

- Select instance x (e.g., a review sentence) to explain locally
- Generate N perturbed samples by randomly masking/replacing tokens
- Query the black-box model for predictions on all N perturbations
- **Weight perturbed samples by their cosine similarity to original x**
- Fit sparse linear model on (perturbed samples, predictions, weights)
- Top- K feature weights of the linear model are the explanation

LIME for Text (LimeTextExplainer)

- Tokenizes text; perturbations remove word groups (bag-of-words space)
- Each word is treated as a binary feature: present (1) or absent (0)
- Positive weight $\rightarrow$ word pushes prediction toward positive class
- Negative weight $\rightarrow$ word pushes prediction toward negative class
- Output: ranked word list with signed weights and highlighted HTML
- `num_features` parameter: controls explanation sparsity (default=10)

Strengths & Limitations

- Pro: completely model-agnostic — works with any classifier
- Pro: produces intuitive word-level highlights for non-experts
- Pro: easy to implement with the lime Python library
- Con: sampling-based — explanations vary slightly across runs
- Con: local only — does not reflect global model behavior
- Con: bag-of-words perturbation ignores word order and context

Case Study: K-Means — Customer Segmentation (Mall Dataset)

Real-world application: segment mall customers by Age, Annual Income, and Spending Score to derive actionable marketing personas.

Dataset & Experiment Setup

- Dataset: Mall Customer dataset — 200 customers, 5 features
- Features used: Annual Income (k\$) and Spending Score (1-100)
- Preprocessing: StandardScaler — zero mean, unit variance
- K selection: Elbow method → K=5 gives the sharpest WCSS bend
- Algorithm: K-Means++ initialization, max_iter=300, n_init=10
- Tool: scikit-learn KMeans; visualization via matplotlib + seaborn

Cluster Profiles Discovered

- Cluster 0 — Careful Spenders: High income, Low spending score → save-oriented
- Cluster 1 — Standard Customers: Mid income, Mid spending → mainstream segment
- Cluster 2 — Target Customers: High income, High spending → premium marketing
- Cluster 3 — Careless Spenders: Low income, High spending → impulse buyers
- Cluster 4 — Frugal Customers: Low income, Low spending → budget-conscious
- Explanation: surrogate decision tree achieves 97% accuracy on cluster labels

Key Insight: Cluster 2 (High Income + High Spending) is the ideal marketing target. The surrogate decision tree rule: IF Income > 70k AND Spending > 60 THEN Cluster 2. SHAP analysis confirms Income contributes 58% and Spending 42% to membership.

Python Implementation: K-Means Explainer

Complete implementation with scikit-learn, surrogate decision tree explanation, silhouette analysis, and cluster visualization.

```
# — Step 1: Load & preprocess —————
import pandas as pd, numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

df = pd.read_csv('Mall_Customers.csv')
X = df[['Annual Income (k$)', 'Spending Score (1-100)']].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# — Step 2: Elbow method —————
wcss = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, init='k-means++',
                n_init=10, random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
plt.plot(range(2,11), wcss, 'bo-')
plt.xlabel('K'); plt.ylabel('WCSS')
plt.title('Elbow Plot'); plt.show()

# — Step 3: Fit final model (K=5) —————
kmeans = KMeans(n_clusters=5, init='k-means++',
                 n_init=10, random_state=42)
labels = kmeans.fit_predict(X_scaled)
sil = silhouette_score(X_scaled, labels)
print(f'Silhouette Score: {sil:.3f}')
```

```
# — Step 4: Surrogate decision tree —————
dt = DecisionTreeClassifier(max_depth=4,
                            random_state=42)

dt.fit(X_scaled, labels)
acc = dt.score(X_scaled, labels)
print(f'Surrogate accuracy: {acc:.2%}')

# — Step 5: Print if-then rules —————
from sklearn.tree import export_text
rules = export_text(dt,
                    feature_names=['Income', 'Spending'])
print(rules)

# — Step 6: Cluster profile heatmap —————
import seaborn as sns
df['Cluster'] = labels
profile = df.groupby('Cluster')[
    ['Annual Income (k$)',
     'Spending Score (1-100)']].mean()
sns.heatmap(profile, annot=True, fmt='.1f',
            cmap='YlOrRd')
plt.title('Cluster Feature Profiles')
plt.show()

# — Step 7: Scatter visualization —————
colors = ['#e74c3c', '#3498db', '#2ecc71',
          '#f39c12', '#9b59b6']
for i in range(5):
    mask = labels == i
    plt.scatter(X[mask,0], X[mask,1],
```

Case Study: LIME for Sentiment Analysis — IMDb Reviews

Explaining a fine-tuned DistilBERT model's sentiment predictions on IMDb movie reviews using LIME token-level attribution.

Experiment Setup

- Dataset: IMDb Large Movie Review dataset — 50,000 reviews (25k/25k split)
- Model: DistilBERT fine-tuned on IMDb → 92.4% test accuracy
- Explainer: LimeTextExplainer, num_features=10, num_samples=2000
- Positive review input: 'The cinematography was breathtaking and the acting superb'
- Negative review input: 'Boring plot, terrible dialogue, complete waste of time'
- LIME runs 3x per instance; explanations consistent (weight variance < 0.05)

Findings & Interpretation

- Positive review: 'breathtaking' (+0.42), 'superb' (+0.38), 'cinematography' (+0.21)
- Negative review: 'boring' (-0.45), 'terrible' (-0.39), 'waste' (-0.31)
- Neutral words ('the', 'and', 'was') receive near-zero attribution weights
- Negation test: 'not breathtaking' flips top word attribution to -0.38 (correctly)
- LIME reveals model relies heavily on single strong sentiment words
- Sarcasm failure: 'Oh great, another terrible film' → wrongly classified as Positive

Key Insight: LIME confirms DistilBERT uses contextually appropriate sentiment words as primary signals. However, sarcasm edge cases expose a weakness: the model fails when sentiment polarity is inverted at a discourse level. Recommendation: augment training data with sarcastic examples and validate with contrastive explanations.

End-to-end implementation using HuggingFace DistilBERT + LIME LimeTextExplainer with explanation visualization and stability analysis.

Model Loading, Wrapper & Explainer Setup

```
# — Step 1: Install & import —————
# pip install lime transformers torch
from transformers import pipeline
from lime.lime_text import LimeTextExplainer
import numpy as np

# — Step 2: Load DistilBERT sentiment model —
clf = pipeline('sentiment-analysis',
               model='distilbert-base-uncased-finetuned-sst-2-english',
               device=-1) # use GPU: device=0

# — Step 3: Wrapper for LIME (needs predict_proba) —
def predict_proba(texts):
    results = clf(texts, truncation=True,
                  max_length=512)
    probs = []
    for r in results:
        if r['label'] == 'POSITIVE':
            probs.append([1-r['score'], r['score']])
        else:
            probs.append([r['score'], 1-r['score']])
    return np.array(probs)

# — Step 4: Create LIME explainer —————
explainer = LimeTextExplainer(
    class_names=['NEGATIVE', 'POSITIVE'],
    bow=True # bag-of-words perturbation
)
```

Instance Explanation, Visualization & Stability Testing

```
# — Step 5: Explain a positive review —————
text_pos = ('The cinematography was breathtaking'
            ' and the acting superb')
exp_pos = explainer.explain_instance(
    text_pos, predict_proba,
    num_features=10,
    num_samples=2000,
    labels=[1] # explain POSITIVE class
)
print(exp_pos.as_list(label=1))
exp_pos.show_in_notebook(text=True) # HTML viz

# — Step 6: Explain a negative review —————
text_neg = ('Boring plot, terrible dialogue,'
            ' complete waste of time')
exp_neg = explainer.explain_instance(
    text_neg, predict_proba,
    num_features=10, num_samples=2000,
    labels=[0] # explain NEGATIVE class
)
exp_neg.save_to_file('lime_negative.html')

# — Step 7: Stability check —————
weights_runs = []
for _ in range(5):
    exp = explainer.explain_instance(
        text_pos, predict_proba,
        num_features=5, num_samples=1000)
    weights_runs.append(
        dict(exp.as_list(label=1)))
import pandas as pd
df = pd.DataFrame(weights_runs)
print('Std dev across runs:')
print(df.std().sort_values(ascending=False))
```